

CI/CD con enfoque DevSecOps — Caso Forseti

Informe técnico — ISWZ3205 Procesos de Software

Hernán Jurado

2026-07-17

- 1 1. Objetivo del proyecto
- 2 2. Contexto del caso de estudio: Forseti
 - 2.1 2.1 Stack objetivo
- 3 3. Arquitectura del pipeline
 - 3.1 3.1 Capa 1 — CI rápido (GitHub Actions)
 - 3.2 3.2 Capa 2 — Release engineering pesado (Jenkins)
 - 3.3 3.3 Capa 3 — GitOps + Kubernetes (Argo CD + kind + Kyverno)
 - 3.4 3.4 Diagrama end-to-end
- 4 4. Herramientas utilizadas y propósito
 - 4.1 4.1 Etapa de CI (feedback en PR)
 - 4.2 4.2 Etapa de release (post-merge)
 - 4.3 4.3 Etapa de despliegue
 - 4.4 4.4 Herramientas evaluadas y **descartadas** — y por qué
- 5 5. Etapas del pipeline en detalle
 - 5.1 5.1 GitHub Actions — `ci.yml` (feedback rápido)
 - 5.2 5.2 CodeQL — SAST
 - 5.3 5.3 OWASP Dependency-Check — SCA
 - 5.4 5.4 Gitleaks — secrets scan
 - 5.5 5.5 `docker-build.yml` — supply chain security
 - 5.6 5.6 Jenkins — release pipeline
 - 5.7 5.7 Argo CD — GitOps sync
 - 5.8 5.8 Kyverno — admission control
 - 5.9 5.9 OWASP ZAP — DAST
- 6 6. Evidencias y resultados
 - 6.1 6.1 GitHub Actions — status matrix
 - 6.2 6.2 Jenkins pipeline — última corrida `main`
 - 6.3 6.3 Kyverno — policies activas
 - 6.4 6.4 Argo CD — health
 - 6.5 6.5 Métricas globales del pipeline
- 7 7. Conclusiones y aprendizajes

7.1	7.1 Sobre la arquitectura elegida
7.2	7.2 Sobre DevSecOps práctico
7.3	7.3 Sobre GitOps
7.4	7.4 Sobre la firma de imágenes
7.5	7.5 Lecciones que quedan
7.6	7.6 Trabajo futuro
8	8. Referencias

1 1. Objetivo del proyecto

El objetivo de este trabajo es implementar y documentar un **pipeline CI/CD con enfoque DevSecOps** completo, aplicado a **Forseti**: un SaaS real de facturación electrónica que emite comprobantes firmados con XAdES-BES contra el Servicio de Rentas Internas (SRI) de Ecuador.

El pipeline debe cubrir el ciclo entero — desde el commit del desarrollador hasta el despliegue en un cluster Kubernetes — integrando en cada etapa controles de seguridad de tipo **SAST**, **DAST**, **SCA**, **secretos**, **firma de artefactos** y **policy-as-code**, sin duplicar la responsabilidad de ninguna herramienta.

Los objetivos específicos son:

1. Diseñar un pipeline con **responsabilidades claramente separadas** entre GitHub Actions (feedback rápido en PR) y Jenkins (release engineering pesado en `main`).
2. Integrar como mínimo **cinco herramientas de análisis de seguridad** que se complementen entre sí — no que se pisen.
3. **Firmar los artefactos** (imágenes Docker + SBOM) con Cosign / Sigstore y **verificar la firma en el cluster** mediante Kyverno.
4. Adoptar el patrón **GitOps** con Argo CD para que el estado del cluster refleje siempre lo que hay en un repositorio Git — sin `kubectl apply manual`.
5. Demostrar el flujo end-to-end sobre un cluster **kind** local reproducible, con manifests y scripts que cualquiera pueda ejecutar.

2 2. Contexto del caso de estudio: Forseti

Forseti es un producto real de **TARIUS S.A.S.** en producción desde junio de 2026. Está desplegado sobre un VPS Vultr Miami con Docker Compose + Nginx + Let's Encrypt y ya emitió comprobantes autorizados por el SRI en ambientes de pruebas y de producción. El código real es privado; para este trabajo académico se generó una **versión demo sanitizada** (repos públicos `forseti-devsecops` y `forseti-devsecops-gitops`) que preserva la lógica sensible del dominio

(firma XAdES-BES, generación de clave de acceso, plantillas XML del SRI) pero **elimina secretos**: `.env` reales, `.p12` de firma electrónica, contraseñas de base de datos y la llave maestra AES-256-GCM que cifra los certificados.

Elegí Forseti como caso porque cumple tres condiciones que refuerzan el ejercicio DevSecOps:

- **Alto riesgo de compliance:** procesa RUCs, correos, direcciones — datos personales bajo la LOPDP (Ley Orgánica de Protección de Datos Personales, Ecuador 2021).
- **Alta sensibilidad criptográfica:** maneja certificados digitales `.p12` privados y firma XAdES-BES; una falla de firma corrompe la validez fiscal del comprobante.
- **Alta cadencia de entrega:** la base tributaria cambia dos veces al año — el pipeline debe soportar releases seguros sin pausar la producción.

2.1 2.1 Stack objetivo

Capa	Tecnología
Backend	Java 21 + Spring Boot 3.4 + Maven, firma XAdES con Apache Santuario, SOAP con Apache CXF, jobs con JobRunr
Frontend	Vue 3 + Vite + TypeScript + PrimeVue + Pinia
Base de datos	PostgreSQL 16 con Row-Level Security (multi-tenant)
Contenedores	Dockerfile multi-stage → imágenes <code>distroless-like</code> (alpine JRE / nginx-alpine)
Orquestación	Kubernetes (cluster kind local para la demo, k3s en VPS en producción real)
Deploy	Helm chart parametrizable + Argo CD (GitOps)

3 3. Arquitectura del pipeline

El pipeline se organiza en **tres capas** con responsabilidades independientes.

3.1 3.1 Capa 1 — CI rápido (GitHub Actions)

Corre en **cada pull request** y da feedback al desarrollador en menos de 5 minutos. Solo bloquea el merge si algo se rompe; no toca imágenes ni cluster.

- `ci.yml` — build Maven + tests JUnit + ArchUnit para backend; `vue-tsc` + ESLint + Vitest + build Vite para frontend.

- `codeql.yml` — SAST oficial de GitHub para Java y TypeScript, con perfil `security-extended + security-and-quality`. Resultados publicados como *code scanning alerts*.
- `dependency-check.yml` — OWASP Dependency-Check sobre Maven + npm, con umbral `--failOnCVSS 8`. SARIF sube a code scanning.
- `gitleaks.yml` — escaneo del historial completo (`fetch-depth: 0`) contra patrones de secretos, con allowlist declarada en `.gitleaks.toml`.
- `docker-build.yml` — solo en `push` a `main`: build de las imágenes con Buildx, scan con Trivy, generación de SBOM con Syft/Anchore, firma **keyless** con Cosign (identidad OIDC del workflow) y `cosign verify` como gate.

3.2 3.2 Capa 2 — Release engineering pesado (Jenkins)

Corre después del merge a `main`. Complementa a GitHub Actions haciendo la parte que Actions no debería hacer (o hace peor):

- Build **reproducible** de imágenes Docker con labels OCI de trazabilidad.
- Doble escaneo Trivy: un pase para reporte SARIF (no bloquea) y un pase con `--severity CRITICAL --exit-code 1` como *gate* explícito.
- SBOM SPDX con Syft — el mismo formato que usa la NIST y la Executive Order 14028 en EE.UU.
- Firma **con llave propia** (`cosign sign --key`) más `attest` del SBOM, además de la firma `keyless` que ya hizo Actions. La política de admisión Kyverno acepta **cualquiera de los dos**, dando redundancia.
- OWASP ZAP baseline scan contra el ambiente `staging` desplegado por Argo CD.
- **Bump del tag en el repo GitOps separado** (`forseti-devsecops-gitops`), que es el paso que dispara el sync de Argo CD.

3.3 3.3 Capa 3 — GitOps + Kubernetes (Argo CD + kind + Kyverno)

El cluster **no acepta cambios directos**. Todo lo que corre en el cluster tiene que venir de un commit al repo GitOps. Argo CD observa el repo cada 3 minutos, detecta el nuevo tag, hace `helm template` con los `values.yaml` del ambiente correspondiente y aplica los manifests con `ServerSideApply`.

Kyverno actúa como **admission controller** — cualquier pod que se intente crear en un namespace de Forseti (`staging` o `prod`) pasa por 5 validaciones antes de ser admitido. Si alguna falla, la API de K8s rechaza el pod con un mensaje descriptivo. Esto blindo al cluster incluso contra un desarrollador que haga `kubectl apply -f pod.yaml` a mano por fuera del pipeline.

3.4 3.4 Diagrama end-to-end

Ver `README.md` sección 2. Resumen textual del flujo:

```
git push (dev)
  → GH Actions: build + CodeQL + Dep-Check + Gitleaks    (≤5 min, bloquea PR)
  → merge a main
  → GH Actions docker-build.yml: Buildx + Trivy + Syft + Cosign keyless (≤10 min)
  → Jenkins: rebuild + Trivy gate + Syft + Cosign key + ZAP DAST + bump gitops (≤20 min)
  → Argo CD detecta commit → helm template → kubectl apply
  → Kyverno valida (5 policies) → admite o rechaza pod
  → pods corriendo, healthchecks verdes
```

4 4. Herramientas utilizadas y propósito

La consigna pide “al menos dos herramientas de análisis de seguridad (SAST y DAST)”. Este pipeline usa **quince** herramientas en total; a continuación se justifica el rol de cada una y **por qué no se pisa con las demás**.

4.1 4.1 Etapa de CI (feedback en PR)

Herramienta	Categoría	Rol específico
GitHub Actions	CI orquestador	Corre 5 workflows en paralelo. Notifica en el PR.
CodeQL	SAST	Análisis estático de Java y TypeScript. Busca inyección SQL, XSS, path traversal, deserialización insegura, etc. sin ejecutar el código.
OWASP Dependency-Check	SCA (dependencies)	Compara <code>pom.xml</code> y <code>package-lock.json</code> contra la NVD para detectar dependencias con CVEs conocidos.
Gitleaks	Secrets scan	Regex + entropy detection sobre el historial completo del repo. Bloquea PRs que introducen keys, tokens o passwords hardcoded.

4.2 4.2 Etapa de release (post-merge)

Herramienta	Categoría	Rol específico
Jenkins	CD orquestador	Pipeline declarativo con 8 stages. Corre en Docker con Docker-in-Docker montando <code>/var/run/docker.sock</code> .
Docker Buildx	Build	Genera imágenes multi-arch reproducibles con etiquetas OCI (<code>org.opencontainers.image.revision</code> , etc.).
Trivy	SCA (imagen)	Analiza el filesystem de la imagen resultante buscando CVEs de OS (Alpine) y libs Java/Node.
Syft	SBOM	Genera Software Bill of Materials en formato SPDX-JSON. Cumple la Executive Order 14028 y NTIA.
Cosign (Sigstore)	Signing	Firma digital de imagen + attestation del SBOM. Dos modos usados: keyless (OIDC de GitHub, Actions) y key-based (par ECDSA generado con <code>cosign generate-key-pair</code> , Jenkins).
GHCR	Registry	OCI-compliant, gratis, integrado con permisos GitHub. Acepta las firmas Cosign como tags adicionales (formato <code>sha256-...sig</code>).

4.3 4.3 Etapa de despliegue

Herramienta	Categoría	Rol específico
Argo CD	GitOps CD	Reconcilia el estado del cluster con el repo <code>forseti-devsecops-gitops</code> cada 3 min. UI web para inspección visual del drift.
kind	K8s local	Cluster de 3 nodos (1 CP + 2 workers) corriendo cada nodo como contenedor Docker. Se levanta en 90s.
Helm	Templating	Chart parametrizable con <code>values.yaml</code> por ambiente (staging/prod).
Kyverno	Policy-as-Code	Admission controller que aplica 5 policies. A diferencia de OPA/Gatekeeper, Kyverno usa YAML declarativo — no requiere aprender Rego.
OWASP ZAP	DAST	Baseline scan (passive + spider, sin ataques activos) contra el ingress de staging. Reporta headers HTTP faltantes, cookies mal configuradas, disclosure de versiones.

4.4 4.4 Herramientas evaluadas y descartadas — y por qué

Herramienta	Motivo del descarte
GitLab CI	El código vive en GitHub. Sería redundante con GH Actions.
Snyk	Excelente pero requiere licencia paga para uso profesional. Dep-Check + Trivy cubren SCA sin licencia.
Notary v2	Estándar OCI abierto pero con menos tooling. Cosign tiene mejor ergonomía y ecosistema (Sigstore, Fulcio, Rekor).
OPA / Gatekeeper	Requiere aprender Rego. Kyverno hace lo mismo en YAML puro — más rápido de justificar en un TFG.
FluxCD	Equivalente a Argo CD. Se eligió Argo CD por su UI, mejor para demostrar visualmente durante la exposición.
Prometheus / Grafana	Observabilidad ≠ DevSecOps. Fuera del alcance de la consigna.
Ansible / Chef	La configuración del cluster ya es declarativa vía Helm + Argo CD. Meter un configurador imperativo sería regresión.

5 5. Etapas del pipeline en detalle

5.1 5.1 GitHub Actions — ci.yml (feedback rápido)

```
jobs:
  backend:
    steps:
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '21', cache: maven }
      - run: mvn -B verify # incluye tests + ArchUnit
      - uses: actions/upload-artifact@v4
        with: { name: forseti-backend-jar, path: backend/target/*.jar }
```

Salidas: .jar archivado (7 días), reportes JUnit visibles en el PR, badge de status en el README.

5.2 5.2 CodeQL — SAST

Corre en matrix contra `java-kotlin` (build-mode: manual, requiere `mvn package -DskipTests`) y `javascript-typescript` (build-mode: none). El perfil `security-extended` habilita queries adicionales fuera del baseline. Los hallazgos suben al tab **Security** → **Code scanning** del repo.

5.3 5.3 OWASP Dependency-Check — SCA

Corre contra el repo entero con `--failOnCVSS 8`. Usa una `NVD_API_KEY` guardada como secreto para acelerar el fetch de la base (sin key demora ~15 min; con key ~2 min). El SARIF resultante se sube a code scanning y el HTML queda como artifact 30 días.

5.4 5.4 Gitleaks — secrets scan

Usa `.gitleaks.toml` propio con allowlist para:

- `.env.example` (plantillas públicas)
- Archivos de test que contienen fixtures como placeholder
- Una llave AES DEV que **sí** aparece en `docker-compose.yml` como valor por defecto (documentada como “solo dev”)

5.5 5.5 docker-build.yml — supply chain security

El workflow más importante. Cada job (backend / frontend) corre en paralelo y ejecuta:

1. `docker/build-push-action` con `provenance: true, sbom: true` (BuildKit genera atestación in-toto).
2. Trivy scan con SARIF upload a code scanning (categorizado por componente).
3. Anchore `sbom-action` genera SBOM SPDX.
4. `cosign sign --yes` (keyless, usa el `id-token: write` del workflow para OIDC).
5. `cosign attest --predicate sbom.spdx.json --type spdxjson` firma el SBOM como atestación.
6. `cosign verify --certificate-identity-regexp` valida que la firma sea del owner esperado.

5.6 5.6 Jenkins — release pipeline

El `Jenkinsfile` declara 9 stages. Los tres clave:

Stage “Trivy — Gate CRITICAL” — corre `trivy image --severity CRITICAL --exit-code 1`. Si hay al menos una vulnerabilidad CRITICAL *fixable* (con parche disponible), rompe el build. Los CVEs sin fix se ignoran con `--ignore-unfixed` para no bloquear por cosas fuera de nuestro control.

Stage “Cosign — firma + attest SBOM” — usa `withCredentials` para inyectar la llave privada Cosign (guardada como *Secret File* en Jenkins) y su password. Firma imagen y SBOM.

Stage “Promover al repo GitOps” — clona el repo `forseti-devsecops-gitops`, edita `environments/staging/values.yaml` con `sed` para bumppear el tag, commitea y hace `git push`. Ese push es lo que despierta a Argo CD.

5.7 5.7 Argo CD — GitOps sync

`Application` staging tiene `syncPolicy.automated.prune=true, selfHeal=true`. Prod tiene sync manual (safeguard). El chart Helm se renderiza con `helm template` interno de Argo CD y los recursos van con `ServerSideApply` para evitar conflictos con controllers que anotan pods (Kyverno mismo lo hace).

5.8 5.8 Kyverno — admission control

Cinco policies enforced:

1. `verify-forseti-image-signatures` — `verifyImages` con dos attestors: llave propia de Jenkins o identidad OIDC keyless de GitHub Actions. Ambos válidos.
2. `require-resource-limits` — todo container debe declarar `requests` y `limits`.
3. `disallow-latest-tag` — bloquea `:latest` en `forseti-prod`.
4. `disallow-privileged-containers` — Baseline PSS: no privileged, no `hostNetwork`, no `hostPID`, no `hostIPC`.
5. `require-run-as-non-root` — Pod o container `securityContext` debe declarar `runAsNonRoot: true`.

5.9 5.9 OWASP ZAP — DAST

Se ejecuta en el stage post-deploy de Jenkins. Baseline scan (no active) contra `http://localhost:8000` (el ingress del cluster kind). Reporte HTML publicado con `publishHTML` — visible directamente en la UI de Jenkins.

6 6. Evidencias y resultados

Los siguientes bloques son los resultados obtenidos al correr el pipeline completo end-to-end en el ambiente de desarrollo. Las capturas se incluyen en el apéndice `docs/capturas/`.

6.1 6.1 GitHub Actions — status matrix

- ✓ `ci.yml` — 3m 42s. 214 tests JUnit verde. Frontend 2 tests Vitest verde.
- ✓ `codeql.yml` — Java 4m 21s / TypeScript 1m 55s. **o alertas open**.
- ✓ `dependency-check.yml` — 3 hallazgos MEDIUM en libs Java transitivas (no fixable aún); o HIGH/CRITICAL.
- ✓ `gitleaks.yml` — 15s. **o secretos detectados** (allowlist funcionó para `.env.example` y el fixture de tests).
- ✓ `docker-build.yml` — 8m 12s total. 2 imágenes firmadas keyless.

6.2 6.2 Jenkins pipeline — última corrida `main`

- Build imágenes Docker — 4m 18s (dos en paralelo, cache warm).
- Trivy — Scan CVEs — backend: 5 HIGH, 12 MEDIUM. frontend: 2 HIGH, 4 MEDIUM.
- Trivy — Gate CRITICAL — **o CRITICAL fixables** ✓ pasa el gate.
- Syft — SBOM SPDX — backend: 187 componentes. frontend: 342 componentes.
- Cosign — firma + attest — 4 tags nuevos en GHCR: `sig`, `att` para cada imagen.
- OWASP ZAP — 2 WARN (X-Frame-Options + CSP en staging local), **o FAIL**.
- Promover al repo GitOps — 1 commit push, Argo CD sync-eó en 47s.

6.3 6.3 Kyverno — políticas activas

NAME	ACTION	READY	AGE
verify-forseti-image-signatures	enforce	True	2h
require-resource-limits	enforce	True	2h
disallow-latest-tag	enforce	True	2h
disallow-privileged-containers	enforce	True	2h
require-run-as-non-root	enforce	True	2h

Test de admisión rechazada (intencional):

```
kubectyl -n forseti-prod run test --image=nginx:latest
Error from server: admission webhook "validate.kyverno.svc-fail" denied the request:
policy disallow-latest-tag/block-latest: validation error: en forseti-prod está
prohibido el tag :latest o imagen sin tag. Usá sha-... o vX.Y.Z.
```

6.4 6.4 Argo CD — health

App `forseti-staging` : **Synced, Healthy**. Última sync: 2m 14s. Drift detectado: `0` recursos.
Prune: `0` recursos huérfanos.

6.5 Métricas globales del pipeline

Métrica	Valor
Tiempo total desde <code>git push</code> a pod running en staging	≈ 22 min
Tiempo hasta feedback en PR (bloquea merge)	≤ 5 min
Herramientas de seguridad ejecutadas por release	7 (CodeQL, Dep-Check, Gitleaks, Trivy, ZAP, Cosign verify, Kyverno)
CVEs CRITICAL fixables no mitigados	0
Secretos encontrados en historial	0
Cobertura de firma Cosign en pods running	100% (Kyverno rechaza los no firmados)

7 Conclusiones y aprendizajes

7.1 Sobre la arquitectura elegida

La decisión de dividir el pipeline en **GitHub Actions + Jenkins** en vez de usar una sola herramienta fue deliberada y resultó acertada. GH Actions da feedback en 5 minutos al PR — lo que hace que los desarrolladores vean el error mientras el contexto está fresco. Jenkins, en cambio, corre en `main` con etapas de 20+ minutos que serían insoportables en un PR: firma con llave propia, escaneo profundo Trivy, DAST con ZAP, promoción a GitOps. Cada herramienta hace lo que hace mejor.

7.2 Sobre DevSecOps práctico

“**Al menos dos herramientas de seguridad**” fue el mínimo de la consigna, pero el pipeline usa **siete** — no por exhibicionismo, sino porque cada tipo de vulnerabilidad requiere un tipo de análisis distinto:

- **SAST** (CodeQL) atrapa bugs de lógica que compilan y funcionan.
- **SCA** (Dep-Check + Trivy) atrapa CVEs en código que no escribimos.
- **Secrets scan** (Gitleaks) atrapa credenciales mal manejadas por humanos.
- **DAST** (ZAP) atrapa problemas de configuración runtime (headers, cookies, disclosure).
- **Signing** (Cosign + Kyverno) atrapa suplantación de artefactos (supply chain).

Ninguno solo hace de los otros redundante. Este es el argumento central de DevSecOps: **defensa en profundidad**.

7.3 7.3 Sobre GitOps

Adoptar GitOps (Argo CD + repo separado) cambió por completo la ergonomía del deploy. En el estado anterior de Forseti (VPS con `git pull + docker build + systemctl restart`), un fix urgente requería SSH al servidor y coordinación humana. Con Argo CD, el fix urgente es *commit + push al repo GitOps* — el mismo mecanismo que cualquier otro cambio. **La auditoría es automática:** cada estado del cluster corresponde exactamente a un commit en Git. Los rollbacks se hacen con `git revert`.

7.4 7.4 Sobre la firma de imágenes

Cosign keyless (OIDC de GitHub) es la mejor experiencia developer que vi en supply chain security. No hay llave privada que gestionar, no hay rotación, no hay HSM — la “identidad” del firmante es la propia identidad del workflow que corrió, garantizada por Sigstore’s Fulcio y auditada en Rekor. En el mundo académico esto es magia; en el mundo real esto es lo que Google, Kubernetes, Chainguard y Wolfi usan.

7.5 7.5 Lecciones que quedan

- **Un pipeline sin gate no es un pipeline.** Correr las herramientas y publicar reportes sin bloquear el build es “security theater”. Trivy con `--exit-code 1`, Kyverno en `Enforce`, y `cosign verify` que rompe si falla — esos son los que importan.
- **Kubernetes solo aporta si hay policies.** Un cluster K8s vacío es tan inseguro como un `docker run` sin flags. Kyverno cierra el círculo: sin él, la firma Cosign es decorativa.
- **La separación de repos código/gitops es incómoda al principio pero paga.** El primer día uno quisiera todo en un repo. Al segundo commit accidental que mezcla lógica de negocio con bump de tag, se entiende por qué van separados.
- **kind sirve para todo hasta que no.** Para la demo académica es perfecto. Para reproducir el ambiente prod (con Ingress + cert-manager + external-dns + escalado horizontal), hay que ir a un cluster gestionado o a k3s en un VPS $\geq 2\text{GB RAM}$.

7.6 7.6 Trabajo futuro

Cuatro extensiones naturales para el proyecto:

1. **Renovate/Dependabot autónomo** — abrir PR de bumps de deps con tests que validen el efecto.
2. **Runtime security con Falco** — detectar comportamiento anómalo en containers en ejecución (complementa DAST post-mortem).
3. **Sealed-Secrets o External-Secrets-Operator** — hoy los secrets del chart son placeholders; en real deben venir de Vault o del proveedor cloud.

4. **Progressive delivery con Argo Rollouts** — canaries y blue-green nativos, reemplazando el rollout default de K8s.

8 8. Referencias

- OWASP Top 10 (2021) — <https://owasp.org/Top10/>
- Sigstore documentation — <https://docs.sigstore.dev/>
- Kyverno policy library — <https://kyverno.io/policies/>
- OWASP Dependency-Check — <https://owasp.org/www-project-dependency-check/>
- OWASP ZAP baseline — <https://www.zaproxy.org/docs/docker/baseline-scan/>
- Argo CD documentation — <https://argo-cd.readthedocs.io/>
- Kubernetes Pod Security Standards — <https://kubernetes.io/docs/concepts/security/pod-security-standards/>
- NIST SP 800-218 (SSDF) — <https://csrc.nist.gov/publications/detail/sp/800-218/final>
- Executive Order 14028 (SBOM requirement) — <https://www.whitehouse.gov/briefing-room/presidential-actions/2021/05/12/executive-order-on-improving-the-nations-cybersecurity/>